ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Higher-Order Functions Challenge - Part I

Work in groups of 3–4. For each challenge, discuss your approach before coding.
Write your solutions in Haskell. When you're confident in a solution, test it in GHCi.

These challenges draw on the material from Lecture 13 (Part I): `map` , `filter` , `foldr` , `foldl` .

Warm-up

Challenge 1. Fold it: Without looking at the notes, write the type and definition of `foldr` from memory. Then check.

Once you have `foldr` , use it to define each of the following *in a single line*:

```
mySum     :: [Int] -> Int
myProduct :: [Int] -> Int
myLength  :: [a] -> Int
myConcat  :: [[a]] -> [a]       -- flattens a list of lists
```

Checkpoint: `myConcat [[1,2],[3],[4,5]] == [1,2,3,4,5]`

Challenge 2 — Left or right? The two definitions below look similar but behave differently. *Without running them*, predict the output of each:

```
ex1 = foldr (\x acc -> "(" ++ x ++ "+" ++ acc ++ ")") "0" ["a","b","c"]
ex2 = foldl (\acc x -> "(" ++ acc ++ "+" ++ x ++ ")") "0" ["a","b","c"]
```

Write down your predictions, then check in GHCi. Can you draw the corresponding trees?

Core

Challenge 3. The universal fold: The lecture claims that `map` and `filter` can be defined in terms of `foldr`. Your task: define **three** more standard list functions using only `foldr` (no direct recursion, no other library functions):

```
myAny :: (a -> Bool) -> [a] -> Bool    -- True if any element satisfies p
myAll :: (a -> Bool) -> [a] -> Bool    -- True if all elements satisfy p
```

Then try:

```
myTakeWhile :: (a -> Bool) -> [a] -> [a]  -- longest prefix satisfying p
```

Questions: The standard `takeWhile` stops as soon as it finds an element that fails the predicate:

- Does your solution with `foldr` express this?
- Does the fold traverse the entire list?
- Does that depend on the evaluation strategy of the language you use for your implementation?

Challenge 4. Pipeline: Define a function:

```
pipe :: [a -> a] -> a -> a              -- pipe [f1, f2, f3] x == f1 (f2 (f3 x))
```

`pipe` composes a list of functions into a single pipeline, applying the last function first.

Key decisions:

- What should `pipe []` be? What is the identity for function composition?
- What operator combines two functions into one?
- Should you fold from the right or from the left to get the nesting `f1 (f2 (f3 x))`?

Test: `pipe [(+1), (*2), (+3)] 4 == 15` (i.e. `(4+3)*2+1`)

Now define a variant:

```
pipeL :: [a -> a] -> a -> a
```

such that `pipeL [f1, f2, f3] x == f3 (f2 (f1 x))` — the *first* function in the list is applied first.

Question: How are `pipe` and `pipeL` related to `foldr` and `foldl`?

Advanced

Challenge 5. Fold fusion: Consider the following two computations:

```haskell
-- Version A
resultA = sum (map (*2) [1..1000000])

-- Version B
resultB = foldr (\x acc -> 2*x + acc) 0 [1..1000000]
```

1. Convince yourselves that `resultA == resultB`.
2. How many traversals of the list does Version A perform? How many does Version B perform?
3. Show that the equation in 1 is an instance of the **fold-map fusion law**:

$$(\text{foldr}\ g\ b)\ \circ\ (\text{map}\ f) = \text{foldr}\ (g \circ f)\ b$$

   Identify `f`, `g`, and `b`.

Wrap-up

- **The trickiest challenge you solved** — walk through your reasoning.
- **A challenge you couldn't solve** — explain where you got stuck and what you tried. (This is just as valuable.)
- **A surprise** — something you discovered that wasn't in the lecture notes.